

Analyses: Classification with KNN

01/03/2024

Description

k-nearest neighbor analyses of the feature vectors per character sequence (loaded from ~/results/features_combined.csv). The results are stored in ~/results/classifier/.

Load Packages

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages("ggplot2")`.

```
library(dplyr)
library(class)
library(gmodels)
library(caret)
```

Load Data

Load data table with estimated feature values per sequence.

```
# load feature value estimations
estimations.df <- read.csv("~/Github/UpperPalCombinatorics/results/features/features_combined.csv")
head(estimations.df)
```

```
##      subcorpus      sequence num.char huni.chars hrate.chars  ttr.chars
## 1 Aurignacien ||| ///// _ V      12  1.8962406   1.5295563  0.41666667
## 2 Aurignacien      vvvvv         5  0.0000000   0.5711903  0.20000000
## 3 Aurignacien      vvvv vvvv      9  0.5032583   0.9060047  0.22222222
## 4 Aurignacien vvvvvvvvvvvvvvv      14  0.0000000   0.6379550  0.07142857
## 5 Aurignacien      |||           3  0.0000000   0.4308271  0.33333333
## 6 Aurignacien      |||           4  0.0000000   0.5070802  0.25000000
##      rm.chars
## 1 0.6363636
## 2 1.0000000
## 3 0.7500000
## 4 1.0000000
## 5 1.0000000
## 6 1.0000000
```

Select subcorpora for binary classification.

```
# choose subcorpora to compare
unique(estimations.df$subcorpus)
```

```
## [1] "Aurignacien"      "Cuneiform (Uruk V)"  "Cuneiform (Uruk IV)"
## [4] "Cuneiform (Uruk III)" "TeDDi"
```

```
subcorpus1 <- "Cuneiform (Uruk III)"
subcorpus2 <- "Cuneiform (Uruk IV)"
selected <- c(subcorpus1, subcorpus2)
estimations.subset <- estimations.df[estimations.df$subcorpus %in% selected, ]
```

Remove NAs (whole row)

```
estimations.subset <- na.omit(estimations.subset)
```

Center and scale the data

```
# first three columns are meta information, the others are the feature values
# be aware that this needs to be changed in case of other table format
estimations.scaled <- cbind(estimations.subset[, c("subcorpus", "sequence", "num.char")],
                           scale(estimations.subset[, c("huni.chars", "hrate.chars",
                                                         "ttr.chars", "rm.chars")]))
```

Create Training and Test Sets

```
# Generating seed
set.seed(1234)
# Randomly generating our training and test samples with a respective ratio of 2/3 and 1/3
datasample <- sample(2, nrow(estimations.scaled), replace = TRUE, prob = c(0.67, 0.33))

# Generate training set (select rows randomly by using datasample vector)
train <- estimations.scaled[datasample == 1, 1:ncol(estimations.scaled)]
# Generate test set
test <- estimations.scaled[datasample == 2, 1:ncol(estimations.scaled)]
```

Initialize data frame

```
knn.results <- data.frame(k = numeric(0), baseline = numeric(0),
                          accuracy = numeric(0), accuracy.pvalue = numeric(0),
                          precision = numeric(0), recall = numeric(0),
                          f1 = numeric(0))
```

Building knn classifier

```
# choose maximum number of neighbors n
n = 100
# run a loop over different numbers of neighbors up to n
for (k in 1:n){
  # knn estimation of labels
  predictions.knn <- knn(train = train[, c("huni.chars",
                                           "hrate.chars",
```

```

        "ttr.chars",
        "rm.chars")],
    test = test[, c("huni.chars",
        "hrate.chars",
        "ttr.chars",
        "rm.chars")],
    cl = train[, "subcorpus"], k = k)

# model evaluation
# creating a dataframe from known (true) test labels
test.labels <- test[, "subcorpus"]
# combining predicted and known classes
class.comparison <- data.frame(predictions.knn, test.labels)
# giving appropriate column names
names(class.comparison) <- c("predicted", "observed")
# inspecting our results table
head(class.comparison)
# get confusion matrix
cm <- confusionMatrix(data = class.comparison$predicted,
    reference = as.factor(class.comparison$observed))
# print(cm)
# get performance metrics from the output list of confusionMatrix()
baseline <- cm$overall['AccuracyNull']
accuracy <- cm$overall['Accuracy']
accuracy.pvalue <- cm$overall['AccuracyPValue']
f1 <- cm[["byClass"]][["F1"]]
recall <- cm[["byClass"]][["Recall"]]
precision <- cm[["byClass"]][["Precision"]]

# unname and round resulting values
baseline <- round(unname(baseline), 2)
accuracy <- round(unname(accuracy), 2)
accuracy.pvalue <- round(unname(accuracy.pvalue), 2)
f1 <- round(unname(f1), 2)
recall <- round(unname(recall), 2)
precision <- round(unname(precision), 2)

# prepare data frame with results
local.results <- data.frame(k, baseline, accuracy, accuracy.pvalue,
    precision, recall, f1,
    row.names = NULL)
local.results.rounded <- round(local.results, 2)
# print(local.results.rounded)
knn.results <- rbind(knn.results, local.results.rounded)
# give predictions alongside the other information in data frame
knn.predictions.df <- cbind(test, predictions.knn)
}
print(knn.results)

```

```

##      k baseline accuracy accuracy.pvalue precision recall  f1
## 1    1    0.53    0.72              0    0.70    0.73 0.71
## 2    2    0.53    0.72              0    0.69    0.73 0.71
## 3    3    0.53    0.76              0    0.74    0.76 0.75
## 4    4    0.53    0.74              0    0.71    0.76 0.74

```

## 5	5	0.53	0.78	0	0.77	0.76	0.76
## 6	6	0.53	0.78	0	0.76	0.77	0.77
## 7	7	0.53	0.78	0	0.76	0.76	0.76
## 8	8	0.53	0.77	0	0.76	0.75	0.76
## 9	9	0.53	0.78	0	0.77	0.76	0.77
## 10	10	0.53	0.78	0	0.77	0.76	0.76
## 11	11	0.53	0.78	0	0.76	0.77	0.77
## 12	12	0.53	0.77	0	0.76	0.75	0.76
## 13	13	0.53	0.78	0	0.77	0.76	0.77
## 14	14	0.53	0.78	0	0.76	0.76	0.76
## 15	15	0.53	0.78	0	0.77	0.77	0.77
## 16	16	0.53	0.78	0	0.77	0.76	0.76
## 17	17	0.53	0.78	0	0.77	0.77	0.77
## 18	18	0.53	0.78	0	0.77	0.77	0.77
## 19	19	0.53	0.78	0	0.77	0.77	0.77
## 20	20	0.53	0.78	0	0.77	0.76	0.76
## 21	21	0.53	0.79	0	0.78	0.77	0.77
## 22	22	0.53	0.78	0	0.77	0.77	0.77
## 23	23	0.53	0.78	0	0.77	0.76	0.77
## 24	24	0.53	0.78	0	0.77	0.77	0.77
## 25	25	0.53	0.79	0	0.77	0.79	0.78
## 26	26	0.53	0.78	0	0.76	0.77	0.77
## 27	27	0.53	0.78	0	0.77	0.77	0.77
## 28	28	0.53	0.78	0	0.77	0.79	0.78
## 29	29	0.53	0.78	0	0.76	0.77	0.77
## 30	30	0.53	0.78	0	0.76	0.77	0.77
## 31	31	0.53	0.78	0	0.76	0.79	0.77
## 32	32	0.53	0.77	0	0.75	0.78	0.76
## 33	33	0.53	0.77	0	0.75	0.79	0.77
## 34	34	0.53	0.78	0	0.75	0.79	0.77
## 35	35	0.53	0.77	0	0.75	0.79	0.77
## 36	36	0.53	0.77	0	0.75	0.78	0.76
## 37	37	0.53	0.77	0	0.75	0.77	0.76
## 38	38	0.53	0.77	0	0.76	0.76	0.76
## 39	39	0.53	0.77	0	0.75	0.77	0.76
## 40	40	0.53	0.77	0	0.75	0.76	0.76
## 41	41	0.53	0.77	0	0.75	0.76	0.76
## 42	42	0.53	0.77	0	0.75	0.76	0.76
## 43	43	0.53	0.77	0	0.75	0.76	0.76
## 44	44	0.53	0.77	0	0.75	0.77	0.76
## 45	45	0.53	0.77	0	0.75	0.77	0.76
## 46	46	0.53	0.77	0	0.76	0.77	0.76
## 47	47	0.53	0.77	0	0.76	0.77	0.76
## 48	48	0.53	0.77	0	0.76	0.77	0.76
## 49	49	0.53	0.77	0	0.76	0.76	0.76
## 50	50	0.53	0.77	0	0.76	0.76	0.76
## 51	51	0.53	0.77	0	0.76	0.76	0.76
## 52	52	0.53	0.77	0	0.76	0.76	0.76
## 53	53	0.53	0.77	0	0.75	0.76	0.76
## 54	54	0.53	0.77	0	0.75	0.76	0.75
## 55	55	0.53	0.77	0	0.75	0.76	0.76
## 56	56	0.53	0.77	0	0.75	0.76	0.76
## 57	57	0.53	0.77	0	0.75	0.76	0.76
## 58	58	0.53	0.77	0	0.75	0.76	0.76

## 59	59	0.53	0.77	0	0.75	0.76	0.76
## 60	60	0.53	0.77	0	0.75	0.76	0.76
## 61	61	0.53	0.77	0	0.75	0.76	0.76
## 62	62	0.53	0.76	0	0.75	0.76	0.75
## 63	63	0.53	0.77	0	0.75	0.77	0.76
## 64	64	0.53	0.77	0	0.74	0.77	0.76
## 65	65	0.53	0.76	0	0.74	0.76	0.75
## 66	66	0.53	0.76	0	0.74	0.76	0.75
## 67	67	0.53	0.76	0	0.74	0.76	0.75
## 68	68	0.53	0.77	0	0.75	0.77	0.76
## 69	69	0.53	0.77	0	0.74	0.77	0.76
## 70	70	0.53	0.76	0	0.74	0.76	0.75
## 71	71	0.53	0.76	0	0.74	0.76	0.75
## 72	72	0.53	0.76	0	0.74	0.77	0.75
## 73	73	0.53	0.76	0	0.74	0.76	0.75
## 74	74	0.53	0.76	0	0.74	0.76	0.75
## 75	75	0.53	0.76	0	0.74	0.76	0.75
## 76	76	0.53	0.75	0	0.73	0.76	0.75
## 77	77	0.53	0.76	0	0.74	0.76	0.75
## 78	78	0.53	0.76	0	0.74	0.76	0.75
## 79	79	0.53	0.76	0	0.74	0.76	0.75
## 80	80	0.53	0.76	0	0.74	0.76	0.75
## 81	81	0.53	0.75	0	0.73	0.76	0.75
## 82	82	0.53	0.76	0	0.73	0.76	0.75
## 83	83	0.53	0.76	0	0.73	0.76	0.75
## 84	84	0.53	0.76	0	0.73	0.76	0.75
## 85	85	0.53	0.75	0	0.73	0.76	0.75
## 86	86	0.53	0.76	0	0.74	0.76	0.75
## 87	87	0.53	0.76	0	0.74	0.76	0.75
## 88	88	0.53	0.76	0	0.74	0.76	0.75
## 89	89	0.53	0.76	0	0.74	0.76	0.75
## 90	90	0.53	0.76	0	0.73	0.76	0.75
## 91	91	0.53	0.76	0	0.73	0.77	0.75
## 92	92	0.53	0.76	0	0.73	0.77	0.75
## 93	93	0.53	0.76	0	0.74	0.77	0.75
## 94	94	0.53	0.77	0	0.77	0.74	0.75
## 95	95	0.53	0.77	0	0.77	0.74	0.75
## 96	96	0.53	0.77	0	0.77	0.74	0.75
## 97	97	0.53	0.77	0	0.77	0.74	0.75
## 98	98	0.53	0.77	0	0.77	0.74	0.75
## 99	99	0.53	0.77	0	0.76	0.74	0.75
## 100	100	0.53	0.77	0	0.76	0.74	0.75

Write to file.

```
# write results
write.csv(knn.results, file = paste(c("~/Github/UpperPalCombinatorics/results/classifier/KNN/performance",
                                     subcorpus1, "_",
                                     subcorpus2,
                                     ".csv"
                                   ), collapse = ""),
          row.names = F)

# write df with predictions for post-hoc analyses
# note that this writes the predictions only for the last run of the loop, i.e. the highest k
```

```
write.csv(knn.predictions.df, file = paste(c("~/Github/UpperPalCombinatorics/results/classifier/KNN/pre",
                                             subcorpus1, "_",
                                             subcorpus2,
                                             ".csv"
                                             ), collapse = ""),
          row.names = F)
```